

Midterm Exam -1

CS 101 Algorithmic Problem Solving

Fall 2023

Name(s): _____

HU ID (e.g., xy01042): _____

1. Nim Game

In the tranquil town of Blacksburg, Izzah and Haisum, renowned for their keen intellect, found themselves engrossed in the Nim Game. Given a heap of stones represented by an integer n , both players take turns removing 1 to 3 stones from the heap. The player who takes the last stone wins the game. Izzah plays first.

Write a Python function called *can-win*(n) that takes an integer n as input and returns True if Izzah can win the Nim Game assuming both players play optimally, and False otherwise.

Constraints

- $1 \leq n \leq 2^{31} - 1$

Interaction

The input comprises an integer n denoting the number of stones.

The output must be True if Izzah can win the Nim Game assuming both players play optimally, and False otherwise.

Sample

Input	Output
5	True
8	False

In the first case, Izzah can remove 1 stone. No matter how many stones Haisum removes (1, 2, or 3), Izzah can always remove the remaining stones and win the game.

In the second case, no matter how many stones Izzah removes (1, 2, or 3), Haisum can strategically respond to always leave a multiple of 4 stones. Consequently, in the final move, Haisum takes the last stone and wins the game.

Exercise

In the space provided, indicate the outputs for the given inputs.

Input	Output
10	True
25	True

Problem Identification

Briefly explain the underlying problem you identified in the above question that led you to your solution.

Input: n (number of stones in the heap)

Output: 'True' if Izzah wins the game by taking the last stone in optimal moves, 'False' otherwise.

Pseudocode

```
def can_win(n):
    if n%4 == 0:
        return 'False'
    else:
        return 'True'

print(can_win(n))
```

Dry Run

Using any two of the inputs provided in the Exercise section above, dry run your pseudocode in the space below.

Optimal move	Remaining stones
-	8
Izzah removes 3 stones	5
Haisum removes 3 stones	2
Izzah removes 2 stones	0

2. Character Identification

Computers save all characters as numbers. There is a standard code called ASCII that they use to correspond a character with a number. In Python, the ASCII code of a character can be found using the built-in function, `ord`. For example,

```
>>> ord('4')
52
>>> ord('X')
88
>>> ord('h')
104
```

The ASCII codes of numbers and letters are as follows.

Characters	Codes
Upper case letters, A-Z	65-90
Lower case letters, a-z	97-122
Digits, 0-9	48-57
Special characters	all other codes

Your task is to write a function which takes character C as an argument, and prints whether the character is a lower case letter, upper case letter, digit, or special character.

Constraints:

- $C \in \text{Uppercase Letters } (A-Z) = \{A, B, C, \dots, Z\}$
- $C \in \text{Lowercase Letters } (a-z) = \{a, b, c, \dots, z\}$
- $C \in \text{Digits } (0-9) = \{0, 1, 2, \dots, 9\}$
- $C \in \text{Special Characters} = \{\text{characters other than } A-Z, a-z, \text{ and } 0-9\}$

Interaction

The input consists of a single line which contains a string C of length 1.

The output must print whether C is a lower case letter, upper case letter, digit, or special character.

Sample

Input	Output
B	B is an uppercase letter
i	i is a lower case letter
*	* is a special character

In the first case, The ASCII code for B is 66, which is between 65 and 90.

In the second case, The ASCII code for i is 105, which is between 97 and 122.

In the third case, The ASCII code for * is 42, which falls in none of the given ranges.

Exercise

In the space provided, indicate the outputs for the given inputs.

Input	Output
5	5 is a digit
&	& is a special character

Problem Identification

Briefly explain the underlying problem you identified in the above question that led you to your solution.

Input: C (string of length 1)

Output: 'C', followed by whether it is a lower case letter, upper case letter, digit, or special character

Pseudocode

```
def char_identify(c):
    a = ord(c)
    if a >= 48 and a <= 57:
        print(c, 'is a digit')
    elif a >= 65 and a <= 90:
        print(c, 'is an uppercase letter')
    elif a >= 97 and a <= 122:
        print(c, 'is an lowercase letter')
    else:
        print(c, 'is a special character')

char_identify(c)
```

Dry Run

Using any two of the inputs provided in the Exercise section above, dry run your pseudocode in the space below.

- Set character: $c = \text{'\&'}$
- Calculate ASCII value: $a = \text{ord}(\text{'\&'})$, which results in $a = 38$
- Check conditions:
 - Is 38 between 48 and 57? No
 - Is 38 between 65 and 90? No
 - Is 38 between 97 and 122? No
- Execute 'else' statement
- Output: '&' is a special character'

3. Simple symbols

You are working as a software developer at Habib University. The university's computer science department is organizing a programming competition for its students. As a part of the competition, you are tasked to create a function called `SimpleSymbols(s)`.

Habib University takes pride in fostering creativity and innovation among its students. In this context, the function `SimpleSymbols(s)` is crucial to ensure the integrity of the competition. The function needs to determine whether the input sequence provided by the participants is acceptable.

The input sequence, denoted as s , is a combination of $+$ and $=$ symbols, with one or more numbers sandwiched between them. An acceptable sequence is defined as one where:

- Each number in s is surrounded by exactly one $+$ symbol on each side. For example, $+2+$, $+5+$, and $+1+$ are valid patterns, while $++2++$, $=5=$, and $==1+$ are invalid patterns.
- There should not be more than one consecutive $+$ or $=$ symbol in the input sequence. For example, $++$ and $==$ are invalid patterns.

Your task is to create the `SimpleSymbols(s)` function, adhering to Habib University's standards of precision and clarity. If the input sequence meets the criteria, the function should return `True`. Otherwise, it should return `False`.

Constraints

- $\text{len}(s) \geq 1$ is `True`
- $S \in \{N, +, =\}$

Interaction

The input comprises of a string s .

The output must contain 'True' or 'False' based on the specified condition.

Sample

Input	Output
$++5+==++2++==2$	False
$+1+=+=+6+=+9+=+1+=+=+5+$	True

In the first case, the number 2 is not surrounded by plus symbols, and the plus sign appears in a contiguous location.

In the second case, all numbers are enclosed by + symbols, making the sequence acceptable for the competition.

Exercise

In the space provided, indicate the outputs for the given inputs.

Input	Output
<code>=+2+=+2+=+=+2+=+2+=+=+2+=</code>	True
<code>3++7+</code>	False

Problem Identification

Briefly explain the underlying problem you identified in the above question that led you to your solution.

Input: String S

Output: 'True' if each number in s is surrounded by exactly one + symbol on each side, and there is not more than one consecutive + or = symbol in the input sequence, 'False' otherwise.

Pseudocode

```
def simple_symbols(s):
    for i in range(len(s)):
        if s[i].isdigit() and (i == 0 or i == len(s)-1):
            return 'False'
        elif s[i].isdigit():
            if s[i-1] != '+' or s[i+1] != '+':
                return 'False'
        elif s[i] == '+' and i != len(s)-1:
            if s[i+1] == '+':
                return 'False'
        elif s[i] == '=' and i != len(s)-1:
            if s[i+1] == '=':
                return 'False'
    return 'True'

print(simple_symbols(s))
```

Dry Run

Loop Iteration 1: $i = 0$

- $s[0] = '3' \Rightarrow \text{isdigit}() = \text{True}$
- Since the first character is a digit, but it's not surrounded by '+', return 'False'.
- Output: 'False'

Iteration	Character $s[i]$	Output
1	'3'	'False'

4. Word length puzzle

Sara has a strange habit. Whenever someone says a sentence, she is stuck on the last word, trying to count the length of the last word they said. This habit is making her lose her mind.

Write a function `last_word(sentence)`, that computes the length of the last word in a sentence uttered by someone during a conversation with Sara, and returns it to her ear (in this case, the screen) when she calls the function. A word, in this context, is a sequence of characters without spaces.

Make sure to call the main function to print the output.

Constraints:

- $1 \leq \text{sentence.length} \leq 1000$
- `sentence` consists of only English letters and spaces ' '.
- There will be at least one word in `sentence`.

Interaction

Input comprises of a string `sentence`.

Output must be an integer representing the length of the last word in that sentence.

Sample

Input	Output
"Hello World"	5
"this midterm is a piece of cake"	4

In the first case, the length of the last word "World" is returned, which is 5.

In the second case, the length of the last word "cake" is returned, which is 4.

Exercise

In the space provided, indicate the outputs for the given inputs.

Input	Output
"luffy is still joyboy"	6
"fly me to the moon"	4

Problem Identification

Briefly explain the underlying problem you identified in the above question that led you to your solution.

Input: String S

Output: The length of last word in S.

Pseudocode

```
def last_word(sentence):
    count = 0
    for i in range(-1, -(len(sentence)), -1):
        if sentence[i] == ' ':
            break
        count += 1
    return count

print(last_word(sentence))
```

Dry Run

Using any one of the inputs provided in the Exercise section above, dry run your pseudocode in the space below.

i	sentence[i]	Condition	Count
-1	'y'	'y' is not a space, increment count	1
-2	'o'	'o' is not a space, increment count	2
-3	'b'	'b' is not a space, increment count	3
-4	'y'	'y' is not a space, increment count	4
-5	'o'	'o' is not a space, increment count	5
-6	'j'	'j' is not a space, increment count	6
-7	' ' (Space)	' ' is a space, exit the loop	6

5. Floyd's Triangle

Given a positive integer n , your task is to write a function $Floyd(n)$, which prints the first n rows of Floyd's triangle.

Constraints

- $1 \leq n \leq 100$

Interaction

The input comprises a single line containing an integer n .

The output must contain the first n rows of Floyd's triangle indented to the right.

Sample

Input	Expected Output
3	1 2 3 4 5 6 7 8 9 10
5	1 2 3 4 5 6 7 8 9 10 11 12 13 14 15

In the first case, a right aligned floyd's triangle is created with 3 rows.

In the first case, a right aligned floyd's triangle is created with 5 rows.

Proposed Solution

```

1 def Floyd(n):
2     num = 0
3     for i in range(n):
4         for j in range(i):
5             print(i, end = ' ')
6             num +=1
7         print()
8
9     Floyd(n)

```

Dry Run

Using the inputs provided in the Sample section above, dry run the proposed pseudocode in the space below to find out errors.

Step	i	j	num	Printed Output
1	0	-	0	-
2	1	0	1	1
3	2	0, 1	3	2 2
4	3	0, 1, 2	6	3 3 3
5	4	0, 1, 2, 3	10	4 4 4 4

Error Identification

Briefly explain the errors you identified in the proposed code solution. Mention the line number and errors in each line.

- Line 3: This line has a logical error. The inner loop starts from 0 and goes up to $i - 1$. In Floyd's triangle, the first row should have 1 number, the second row should have 2 numbers, and so on. Therefore, the inner loop should go from 0 to i , or the outer loop should go from 1 to $n+1$.
- Line 5: This line has a logical error. It prints the value of i in each iteration of the inner loop. Instead, it should print num because num is supposed to keep track of the numbers in Floyd's triangle.
- Line 2: num is initialized with 0, hence even if we print num instead of i , the first number printed will be 0, which doesn't follow the pattern of Floyd's triangle.

Correct Solution

Rewrite the lines of code you mentioned above with their errors corrected.

```

1  def Floyd(n):
2      num = 1
3      for i in range(n):
4          for j in range (0,i+1):
5              print (num, end = ' ')
6              num +=1
7          print()
8
9  Floyd(n)
```

Rough Work